

Die Programmiersprache Julia

Bachelorseminar Programmiersysteme

B. Sc. Informatik



Name: Vanessa Köbke

E-Mail: vanessa.koebke@gmx.de

Matrikelnummer: 8878390

Lehrgebiet: Programmiersysteme

Betreuer: Robin Stunic

Abgabedatum: 11.08.2026

Fakultät für Mathematik und Informatik

FernUniversität in Hagen

Abstract

Julia is a relatively new programming language which aims to combine the productivity of high-level languages such as Python with the performance of low-level languages such as C or Fortran. Julia is dynamically typed, follows the Multiple Dispatch paradigm and offers meta-programming similar to Lisp. In terms of popularity in May of 2026 Julia ranks on position 33 of the TIOBE Programming Community Index, and has been downloaded over a 100 million times. The language is becoming increasingly popular for scientific computing and data science due to its high-performance built-in algebraic operations.

In the first part, this work presents the key concepts of the Julia programming language version 1.12.6, providing small code examples to illustrate the presented characteristics. In the second part, this work carries out an exploratory run-time comparison with Java, Rust and C++. For this purpose two algorithms to solve discussion-based semantics for abstract argumentation frameworks that were implemented in Java for my bachelor's thesis will be implemented in Julia and their runtimes will be benchmarked. This experiment confirms that even Julia's built-in functions for matrix operations, more specifically matrix-matrix and matrix-vector multiplications, without the use of specific packages or optimizations outperforms the equivalent Java code using the EJML (Efficient Java Matrix Library) and measures well against Rust and C++ implementations.

Selbstständigkeitserklärung

Ich, Vanessa Köbke, erkläre, dass ich die schriftliche Implementierung/Ausarbeitung zum Bachelorseminar Programmiersysteme selbstständig und ohne unzulässige Inanspruchnahme Dritter fertiggestellt habe. Ich habe dabei nur die angegebenen Quellen und Hilfsmittel verwendet und die aus diesen wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht. Die Versicherung selbstständiger Arbeit gilt auch für enthaltene Zeichnungen, Skizzen oder graphische Darstellungen. ChatGPT wurde zur Unterstützung des allgemeinen Verständnisses, Literaturrecherche und dem Debugging der Implementierungen verwendet. Codex wurde zur Übersetzung des Julia-Codes in Rust und C++ verwendet.

Inhaltsverzeichnis

1	Einleitung	1
2	Eigenschaften von Julia	2
2.1	Dynamische Typisierung	2
2.2	Geschwindigkeit	6
2.3	Multiple-Dispatch-Paradigma	7
2.4	Meta-Programmierung	9
2.5	Unterstützte Programmierparadigmen	10
2.6	Parallelisierbarkeit	11
2.7	Kompatibilität mit anderen Programmiersprachen	11
3	Experimentelle Untersuchung der Performanz von Julia	12
3.1	Diskussions-basierte Semantik für Argumentations-Frameworks . .	13
3.2	Methodologie	14
3.3	Algorithmen zur Lösung von STRONGERDIS	15
3.4	Julia-Implementierungen	16
3.5	Ergebnisvalidierung	18
3.6	Laufzeitergebnisse und Diskussion	19
4	Fazit und Ausblick	23
	Literatur	25
A	Anhang	29
A.1	Sequenzielle Java-Implementierung von Algorithmus 1 mit der EJML-Bibliothek	29
A.2	Sequenzielle Java-Implementierung von Algorithmus 2 mit der EJML-Bibliothek	29

Abbildungsverzeichnis

1	Statische Typisierung von Variablen: Java-Code	3
2	Dynamische Typisierung von Variablen: Julia-Code	3
3	Dynamische Typisierung von Methodenparametern: Julia-Code . .	4
4	Deklaration von Datentypen in Julia	5
5	Deklaration von Datentypen in Julia	5
6	Laufzeiten von Matrixmultiplikationen in verschiedenen Program- miersprachen. Entnommen aus [13, S. 54].	8
7	Deklaration von Datentypen in Julia	8
8	Anwendung der Diagonal-Regel	9
9	Beispielhafte Verwendung der Makros @simd, @view und @benchmark	10
10	SIMD-Parallelisierung in Julia	11
11	Ausführung von Python- und C-Code in Julia	12
12	Beispiel eines Abstrakten Argumentations-Frameworks	13
13	Julia-Implementierung von Algorithmus 1	17
14	Julia-Implementierung von Algorithmus 2	18
15	Julia-Code für die Ergebnisvalidierung	19
16	Laufzeitvergleich für Algorithmus 1	20
17	Relative Laufzeit für Algorithmus 1	21
18	Laufzeitvergleich für Algorithmus 2	21
19	Relative Laufzeit für Algorithmus 2	22
20	Java-Implementierung von Algorithmus 1	29
21	Java-Implementierung von Algorithmus 2	30

1 Einleitung

Looks like Python, feels like Lisp and runs like C.

Julia Tutorial, Minute 6 [12]

Julia ist eine in 2012 entwickelte Programmiersprache, deren Ziel es ist, hohe Geschwindigkeit mit einfacher Verwendbarkeit zu verbinden [7, S. 87]. Der oben zitierte Slogan soll ausdrücken, dass Julia die Produktivität und gute Lesbarkeit von Python mit den Konzepten der Meta-Programmierung von Lisp und der Geschwindigkeit von C kombiniert [12]. Julia richtet sich dabei u. a. auch an Menschen mit wenig Programmiererfahrung und -fähigkeiten, z. B. im wissenschaftlichen Umfeld [5, S. 1]. Auf der offiziellen Webseite werden die folgenden Eigenschaften von Julia hervorgehoben [19]:

- Geschwindigkeit
- Dynamische Typisierung
- Reproduzierbarkeit
- Verwendung des Multiple-Dispatch-Paradigmas
- Generalisierung
- Open Source

Darüber hinaus wirbt Julia mit einem breiten Ökosystem aus aktiven Mitarbeitenden und zum Zeitpunkt der Erstellung dieser Arbeit (Mai 2026) über 12.000 zusätzlichen Bibliotheken für Aspekte wie Datenvisualisierung, maschinelles Lernen oder parallele Programmierung [19].

Ziel dieser Arbeit ist es, die wichtigsten Merkmale von Julia darzustellen und die beworbene Geschwindigkeit im Vergleich zu anderen Programmiersprachen anhand eines einfachen Experiments nachzuvollziehen.

2 Eigenschaften von Julia

Die Programmiersprache Julia wurde 2012 von Jeff Bezanson, Stefan Karpinski, Viral B. Shah und Alan Edelman als Open-Source-Projekt unter der MIT-Lizenz vorgestellt [7, S. 87]. Karpinski und Bezanson begannen bereits 2009 ihre Arbeit an Julia und im Jahr 2018 wurde Julia v1.0 veröffentlicht [17, S. 145]. Im Mai 2026 ist die Version 1.12.6 von Julia verfügbar und belegt Platz 33 der populärsten Programmiersprachen nach dem TIOBE Programming Community Index. Damit liegt Julia vor Sprachen wie TypeScript (Platz 35), Lisp (Platz 41) oder Haskell (Platz 47) [22]. Julia kommt allerdings weder im GitHub Innovation Graph [11] (basierend auf GitHub Repositories) noch in der StackOverflow Developer Survey 2025 (basierend auf Umfragen) [21] vor. Die Gründe für diese Diskrepanz sind nicht unmittelbar ersichtlich.

Laut einigen der Entwickler von Julia gab es im Bereich der wissenschaftlichen Programmierung zwei Optionen; einfache Programmierung in leicht lern- und lesbaren Sprachen wie Python, MATLAB oder R oder hohe Performanz mit Sprachen wie C, C++ oder Fortran [5, S. 1]. Das sogenannte Zwei-Sprachen-Problem [17, S. 147] beschreibt die Tatsache, dass wissenschaftliche Anwendungen initial oft in produktiven Sprachen geschrieben werden und später, wenn die Datenmengen oder Komplexität höher werden, mit großem Aufwand in eine performantere Sprache migriert werden müssen. Das Ziel von Julia ist es, den Graben zwischen diesen beiden Optionen zu überbrücken. Dazu verbindet sie Eigenschaften aus Skript-ähnlichen Sprachen (wie dynamische Typisierung und das Multiple-Dispatch-Paradigma), mit Eigenschaften performanter Sprachen (wie typenspezifische Just-in-time-Compiler und Einfluss auf die Speicherstrukturen der Daten) [5, S. 2].

2.1 Dynamische Typisierung

Während statisch-typisierte Sprachen bei der Deklaration einer Variable die Angabe eines Typs fordern und dieser Typ unveränderlich ist, erlauben dynamisch-

```
1 int zahl;  
2 zahl = 1; //Funktioniert  
3 zahl = "Hallo"; //Fehler  
4 zahl = {1, 2, 3}; //Fehler
```

Abbildung 1: Statische Typisierung von Variablen: Java-Code

```
1 zahl = nothing  
2 zahl = 1 #Funktioniert  
3 zahl = "Hallo" #Funktioniert  
4 zahl = [1, 2, 3] #Funktioniert  
5 zahl = [1, "Hallo", [1, 2, 3]] #Funktioniert
```

Abbildung 2: Dynamische Typisierung von Variablen: Julia-Code

typisierte Sprachen die Deklaration von Variablen ohne Typangabe und die Zuweisung von Objekten verschiedener Typen auf diese Variable. Abbildungen 1 und 2 illustrieren ein Minimalbeispiel in Julia und Java. Abbildung 2 zeigt darüber hinaus, dass Julia zum Abschluss einer Anweisung kein Semikolon benötigt und dass auch innerhalb eines Arrays Objekte verschiedener Typen koexistieren können. Weiter zeigt das Schlüsselwort `nothing` an, dass eine Variable deklariert, aber noch nicht initialisiert wurde. Dies ist nicht zu verwechseln mit der Null-Initialisierung in Java; Julia kennt keinen Null-Wert [5, S. 13].

Das gleiche Prinzip gilt ebenfalls für die Signatur einer Funktion; bei der Deklaration müssen keine Typen für die Parameter der Funktion angegeben werden. Dies ist in Abbildung 3 illustriert. Optional können Typangaben allerdings mittels des Operators `::` hinzugefügt werden, wie in den letzten beiden Zeilen von Abbildung 3 illustriert. Die Überprüfung, ob der zugewiesene Wert dem annotierten Typ entspricht, erfolgt erst zur Laufzeit und verursacht einen Laufzeitfehler, falls die Typen nicht übereinstimmen und die automatische Konvertierung fehlschlägt [5, S. 11]. Eine statische Typenprüfung durch den Compiler erfolgt in Julia nicht [25, S. 4]. Dies liegt u. a. daran, dass der Interpreter ggf. dynamisch Pakete nachlädt und daher zur Übersetzungszeit die Typenprüfung fehlschlagen würde [8, S. 12–13].


```
1 function sagHallo(name)
2     println("Hallo_" * string(name))
3 end
4
5 sagHallo("Leser")           #Ausgabe: "Hallo Leser"
6 sagHallo(1)                 #Ausgabe: "Hallo 1"
7
8 zahl::Int64 = 1 #Funktioniert
9 zahl = "Hallo" #Fehler
```

Abbildung 3: Dynamische Typisierung von Methodenparametern: Julia-Code

Des Weiteren ist Abbildung 3 ein Beispiel für die Verwendung des Schlüsselworts `end`. Dies wird verwendet, um beispielsweise das Ende einer Methodendeklaration, `for`-Schleife oder `if`-Anweisung zu markieren, und erklärt die Abwesenheit von Javas geschweiften Klammern oder Pythons Whitespace-Syntax. Ein weiterer Unterschied zu Java ist, dass Bedingungen von Schleifen- oder `if`-Anweisungen nicht in Klammern stehen.

Darüber hinaus verfügt Julia über ein graduelles Typensystem, das es erlaubt untypisierten, Skript-ähnlichen Code aber auch vollständig typannotierten Code zu verwenden und das insbesondere die Interoperabilität zwischen beiden Arten von Code sicherstellt [8, S. 7]. Wird kein expliziter Typ angegeben, verwendet Julia als Default den abstrakten Datentyp `Any`, der der Supertyp aller Typen ist [25, S. 4]. Julia unterscheidet zwischen abstrakten Datentypen, die Subtypen aber keine Attribute haben können, und konkreten Datentypen, die Attribute aber keine Subtypen haben können [25, S. 4]. Das Schlüsselwort `type` deklariert Datentypen ohne Felder, wobei der Modifikator `abstract` für abstrakte Typen steht und `primitive` für konkrete Datentypen ohne Felder [ibid.]. Konkrete Datentypen mit Feldern werden mit dem Schlüsselwort `struct` deklariert und ihr Datentyp ist standardmäßig unveränderlich. Der Modifikator `mutable` macht ein `struct` veränderlich. Dies ist illustriert in Abbildung 4. Damit unterscheidet sich das Typensystem von Julia fundamental von traditionell objektorientierten Programmiersprachen wie Java [18, S. 115–125].

Zusätzlich kennt Julia das Konzept der Vereinigung von Datentypen [25, S. 7].

```
1 abstract type Integer <: Real end #abstrakter Datentyp
   Integer als Subtyp von Real
2
3 primitive type Int64 <: Signed 64 end #konkreter Datentyp
   Int64 als Subtyp von Signed 64
4
5 struct Person #konkreter Datentyp mit Feldern
6     name::String
7     alter::Int
8 end
```

Abbildung 4: Deklaration von Datentypen in Julia

```
1 x::Union{String, Int} = "Hallo" #funktioniert
2 x = 2 #funktioniert
3 x = true #Laufzeit-Fehler
```

Abbildung 5: Deklaration von Datentypen in Julia

Dies ermöglicht es, einer Variable nicht nur einen einzigen Datentyp zuzuweisen, sondern eine Menge an möglichen Datentypen. Abbildung 5 illustriert, wie die Variable `x` vom Datentyp `String` oder `Int` sein kann, nicht aber vom Typ `boolean`. Einen expliziten Intersektionsoperator für Datentypen gibt es nicht, da die Berechnung der Intersektion von zwei beliebigen Datentypen schwer zu berechnen ist und selbst ein Algorithmus, der nur eine Annäherung der Intersektion berechnet, langsam läuft und fehlerbehaftet ist [25, S. 20]. Stattdessen wurde die `simple_meet`-Funktion eingeführt, die die Intersektion für einfache Anwendungsfälle berechnet, allerdings einige offene Probleme nicht löst [ibid.].

Weiterhin unterscheidet Julia – anders als beispielsweise Java – nicht zwischen Wert- und Referenztypen als zwei getrennten Typkategorien [5, S. 7]. Stattdessen kann die Implementierung für konkrete Typen abhängig vom Anwendungsfall entscheiden, ob diese direkt als Wert oder über eine Referenz repräsentiert und verarbeitet werden [ibid.]. Abstrakte Typen werden hingegen grundsätzlich über Referenzen repräsentiert [ibid.].

2.2 Geschwindigkeit

Dynamisch-typisierte Sprachen bezahlen die Flexibilität der dynamischen Typisierung oft mit einem Geschwindigkeitsverlust von mindestens einer Größenordnung im Vergleich zu statisch-typisierten Sprachen [5, S. 2]. Damit Julia trotz dynamischer Typisierung hohe Geschwindigkeit erreichen kann, sind laut den Entwicklern drei Aspekte von Bedeutung [5, S. 2–3].

- Sprach-Design
Auch wenn Julia dynamisch typisiert ist, erlaubt die Sprache optionale Typendeklarationen in Variablen und Methodenparametern [5, S. 2].
- Optimierung durch den Compiler
Der Julia-Compiler löst überladene Methoden auf und weist jedem Aufruf die Methode mit der spezifischsten passenden Signatur zu [5, S. 2]. Zu diesem Zweck führt der Compiler eine Datenflussanalyse durch, um den Datentyp einer Variable zu erkennen [5, S. 11]. Weiter führt Julias Compiler automatisches Objekt-Unboxing durch [5, S. 2] und das Inlining von Funktionsaufrufen, d. h. das Ersetzen eines Funktionsaufrufs durch den Rumpf der aufgerufenen Methode [5, S. 13].
- Programmierstil
Für optimale Performanz wird empfohlen, Gebrauch von der Typannotation zu machen und Methoden so zu implementieren, dass jeder Variable ein Typ zugeordnet werden kann [5, S. 3]. Bezanson et al. zeigen, dass tatsächlich 63% des analysierten Julia-Codes vollständig typ-annotiert ist [5, S. 14].

Darüber hinaus inkorporiert Julia effiziente Fortran- und C-Bibliotheken für Funktionalitäten der linearen Algebra [17, S. 144].

Bezanson et al. analysieren die Laufzeit für 10 kleine Programme aus dem programming language benchmark game gegen die gleichen Programme in JavaScript, Python und C [5, S. 5]. Julia läuft konsistent schneller als JavaScript und

Python und braucht in etwa die doppelte Laufzeit des äquivalenten C-Programms [ibid.]. Diese Tatsache schreiben die Autoren hauptsächlich der Speicherverwaltung zu, die in Julia durch den Garbage Collector erfolgt und nicht explizit vom Programmier verwaltet wird [ibid.].

Januszek und Pleszczyński analysieren die Laufzeit der Multiplikation von Matrizen der Dimension 1 bis 100 in den Programmiersprachen Wolfram, Python, Julia, R, C# [13]. Matrixmultiplikation hat eine Laufzeitkomplexität von $O(n^3)$ [13, S. 54]. Zu erwarten ist daher eine parabelartige Kurve. Dies bestätigt sich für alle getesteten Programmiersprachen, bis auf Julia, die visuell interpretiert eine konstante Laufzeit im Vergleich zu den anderen Sprachen aufweist (siehe Abbildung 6). Die naheliegende Vermutung ist, dass die Matrixmultiplikation in Julia ebenfalls einer parabelförmigen Kurve folgt, allerdings deutlich langsamer wächst, sodass die Laufzeit für die Dimensionen 1 bis 100 konstant erscheint. Insbesondere auf heutigen hochperformanten Rechnern ist eine Eingabegröße von 100 zu vernachlässigen. Selbst in iterierten Matrixmultiplikationen mit $O(n^4)$ in Java ist die Worst-Case-Laufzeit unter einer Sekunde, der Median sogar unter $100\ \mu s$ [16]. Dass ein Schaubild wie das in Abbildung 6 entsteht, ist möglicherweise einer weniger performanten Hardware aus 2018 geschuldet oder aber dem Vergleich mit wenig performanten Sprachen wie Python. Die zweite Hypothese wird in Abschnitt 3 genauer untersucht, indem ein Laufzeitvergleich mit den Sprachen Java, Rust und C++ durchgeführt wird.

2.3 Multiple-Dispatch-Paradigma

Das Prinzip von Multiple Dispatch besagt, dass jede Funktion beliebig oft überladen sein kann [5, S. 8]. Dabei wird jede konkrete Implementierung der Funktion mit einer spezifischen Signatur als Methode bezeichnet [ibid.]. Bei einem Funktionsaufruf weist Julia jedem Aufruf die Methode mit der spezifischsten passenden Signatur zu [ibid.]. Ein Beispiel ist der Operator `*`; wie in Abbildung 7 illustriert führt dieser Operator eine Konkatenation aus, wenn er auf zwei String-Werte angewendet wird. Wird er hingegen auf zwei Zahlenwerte angewendet, führt er eine

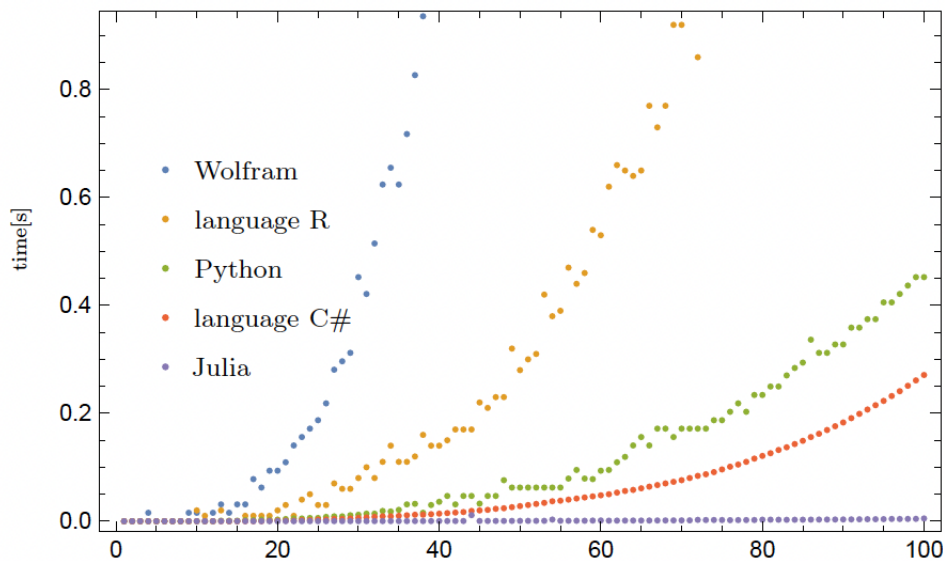


Abbildung 6: Laufzeiten von Matrixmultiplikationen in verschiedenen Programmiersprachen. Entnommen aus [13, S. 54].

```

1 println("Hallo" * "Welt")           #Ausgabe "Hallo Welt"
2 println(3 * 4)                       #Ausgabe 12

```

Abbildung 7: Deklaration von Datentypen in Julia

Multiplikation aus. Der Unterschied zum Single-Dispatch-Paradigma ist, dass bei Multiple Dispatch die vorhandenen Argumente gleich behandelt werden [5, S. 9].

Subtyp-Beziehungen werden in Julia mit dem Operator `<:` ausgedrückt [5, S. 7]. Weiter gilt bei der Verwendung von Typparametern die Diagonal-Regel, d. h. wenn ein Typparameter mehrfach in einer Methode vorkommt, muss er jedes Mal mit dem gleichen Typ belegt sein [25, S. 2]. Abbildung 8 illustriert diese Regel; die Argumente von `f` müssen Subtypen von `Number` sein, allerdings der gleiche Subtyp. Aus diesem Grund funktioniert der Aufruf mit zwei `Int64`-Argumenten und mit zwei `Float64`-Argumenten, nicht aber mit einer Kombination aus beiden.

```
1 function f(x::T, y::T) where T<:Number
2     return x * y
3 end
4
5 println(f(2, 3))           # funktioniert
6 println(f(2.0, 3.5))      # funktioniert
7 println(f(2, 2.5))        # Laufzeit-Fehler
```

Abbildung 8: Anwendung der Diagonal-Regel

2.4 Meta-Programmierung

Julia beinhaltet verschiedene Möglichkeiten der Meta-Programmierung, d. h. Techniken, bei denen das Programm Code erzeugt oder vor der Ausführung transformiert. In Julia wird dies insbesondere durch Makros umgesetzt, die den Quellcode vor der Kompilierung manipulieren und dadurch Optimierungen oder syntaktische Abkürzungen ermöglichen. Makros erlauben es ein Argumenten-Tupel auf einen Ausdruck abzubilden, die ausgeführt werden bevor der eigentliche Code ausgeführt wird, d. h. Makros erhalten den Quellcode in Form syntaktischer Ausdrücke als Eingabe und erzeugen daraus einen neuen Ausdruck, der anschließend kompiliert und ausgeführt wird [5, S. 9]. Dadurch kann Code bereits vor der eigentlichen Programmausführung automatisch transformiert oder erweitert werden [ibid.]. Makros sind eine nützliche Möglichkeit, um beispielsweise Laufzeitanalysen durchzuführen oder dem Compiler Anweisungen zu geben, wie der Code auszuführen ist (z. B. parallelisierte Ausführung).

Abbildung 9 illustriert die Verwendung einiger Makros. Das Makro `@simd` signalisiert dem Julia-Compiler, dass die Operationen der `for`-Schleife vektorisiert werden können, d. h. mehrere Operationen können parallel ausgeführt werden [4, S. 32]. Das `@view`-Makro in Zeile 14 signalisiert dem Julia-Compiler, dass es die Spalte 1 der Matrix für die Summenfunktion nicht kopieren, sondern lediglich lesend darauf zugreifen soll [20]. Das `@benchmark`-Makro aus dem `BenchmarkTools`-Paket kann dazu verwendet werden, um die Laufzeit einer Funktion zu messen. Dazu führt das Makro die Funktion mehrfach aus zur JIT-Optimierung und führt danach die eigentlichen Messungen durch. Anstatt das

Ergebnis der Funktion zurückzugeben, gibt das Makro Werte wie die minimale und maximale Ausführungszeit, die durchschnittliche und Median-Laufzeit aus [3].

```
1 using BenchmarkTools
2
3 function matrix_sum(A)      # summiert alle Zellen einer
    Matrix
4     s = 0.0
5
6     @simd for i in eachindex(A)
7         s += A[i]
8     end
9
10    return s
11 end
12
13 A = rand(10,10)           # erstellt eine 10 x 10 Zufalls-Matrix
14 summe_spalte = sum(@view A[:, 1])
15 @benchmark matrix_sum(A)
```

Abbildung 9: Beispielhafte Verwendung der Makros @simd, @view und @benchmark

2.5 Unterstützte Programmierparadigmen

Obwohl Julia bis zu einem gewissen Grad das Konzept von abstrakten und konkreten Klassen (abstract type und struct) und Subtypen kennt, halten Bezanon et al. fest, dass Julia sich nicht zur objektorientierten Programmierung eignet, da der Sprache die OOP-typische Kapselungsmechanik von Klassen fehlt, d. h. alle Attribute einer Klasse sind immer öffentlich [5, S. 10].

Julia unterstützt funktionale Programmierung durch höherwertige Funktionen, d. h. Funktionen, die andere Funktionen als Argumente nehmen [5, S. 11].

```
1 v1 = [1, 2, 3]
2 v2 = [4, 5, 6]
3 result = Vector{Int}(undef, 3)
4
5 @simd for i in 1:3
6     result[i] = v1[i] + v2[i]
7 end
```

Abbildung 10: SIMD-Parallelisierung in Julia

2.6 Parallelisierbarkeit

Julia ermöglicht Parallelisierung über asynchrone, verteilte oder multi-thread Berechnungen [17, S. 148]. Gevorkyan et al. vergleichen in ihrer Arbeit die Parallelisierbarkeit von Vektoradditionen und deren Effizienz in den Programmiersprachen Julia und Fortran [10]. Dabei verwenden sie die SIMD-Funktionalität (single instruction stream, multiple data stream) über das @simd-Makro, die moderne Prozessoren zur Verfügung stellen [10, S. 4].

Julia erlaubt mittels des @simd-Makros (siehe Abschnitt 2.4) explizit Gebrauch von dieser Funktionalität zu machen [10, S. 5]. Abbildung 10 illustriert dies an dem Beispiel der Vektoraddition. Jede Zeile des Ergebnisvektors wird unabhängig von den anderen Zeilen berechnet. Daher können die Berechnungen in der for-Schleife parallel ausgeführt werden.

2.7 Kompatibilität mit anderen Programmiersprachen

Julia beinhaltet Funktionalitäten, um Code in anderen Programmiersprachen wie C, Fortran oder Python auszuführen oder deren Bibliotheken zu verwenden [17, S. 145, 147]. C-Code kann über die Funktion `ccall` der Julia-Standardbibliothek und Python über die Bibliothek `PyCall` ausgeführt werden. Abbildung 11 illustriert dies.


```
1 using PyCall
2 py"print('Hallo aus Python')"
3
4 ccall(:puts, Cint, (Cstring,), "Hallo aus C")
```

Abbildung 11: Ausführung von Python- und C-Code in Julia

3 Experimentelle Untersuchung der Performanz von Julia

Julia wirbt mit seiner hohen Geschwindigkeit und Möglichkeiten der Parallelisierung im Bereich numerischer Berechnungen. Wie in Abschnitt 2.2 beschrieben, weist Julia bei Matrixmultiplikationen bis zu einer Dimension von 100 eine visuell nahezu konstante Laufzeit im Vergleich mit anderen Programmiersprachen auf. Im Rahmen meiner Bachelor-Arbeit [16] analysiere ich die Laufzeit verschiedener Algorithmen zum Vergleich von zwei Argumenten innerhalb eines Argumentations-Frameworks in der Programmiersprache Java. Diese Algorithmen basieren auf Matrixmultiplikationen und Matrix-Vektor-Multiplikationen. Damit stellen diese Algorithmen nicht nur ein Problem der linearen Algebra dar, sondern eignen sich aufgrund der Natur von Matrix- bzw. Matrix-Vektor-Multiplikationen inhärent zur Parallelisierung [14]. Aus diesem Grund erscheinen diese Algorithmen ein interessanter Testfall zu sein, um Julias Geschwindigkeit gegen die äquivalenter Implementierungen in anderen Programmiersprachen zu testen. In diesem Abschnitt werden die entsprechenden Algorithmen in Julia, Rust und C++ implementiert und die Laufzeiten mit den Java-Implementierungen aus meiner Bachelorarbeit [16] verglichen. Rust und C++ wurden auf Anregung von Herrn Stunic in den Vergleich aufgenommen, da diese beiden Sprachen nativen Code erzeugen und für ihre hohe Geschwindigkeit bekannt sind.

1. Argument 1: Der Himmel ist blau, also kann es nicht regnen.
2. Argument 2: Wasser fällt herab, also muss es regnen.
3. Argument 3: Der Rasensprenkler des Nachbarn ist angeschalten.

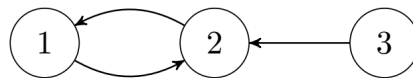


Abbildung 12: Beispiel eines Abstrakten Argumentations-Frameworks

3.1 Diskussions-basierte Semantik für Argumentations-Frameworks

Abstrakte Argumentations-Frameworks nach Dung [9] bestehen aus einer Menge von Argumenten und einer Menge an Angriffsbeziehungen zwischen diesen Argumenten. Abbildung 12 stellt ein beispielhaftes Argumentations-Framework dar. Es besteht aus drei Argumenten, wobei sich Argumente 1 und 2 gegenseitig angreifen (sie widersprechen sich) und das dritte Argument ebenfalls das zweite Argument angreift.

Abstrakte Argumentations-Frameworks können als gerichtete Graphen dargestellt werden, wobei Argumente die Knoten und Angriffsbeziehungen gerichtete Kanten sind. Das abstrakte Argumentations-Framework aus dem Beispiel kann wie folgt dargestellt werden: $G = (\{1, 2, 3\}, \{(1, 2), (2, 1), (3, 2)\})$.

Abstrakte Argumentations-Frameworks werden in der Analyse von Social-Media-Diskussionen, der automatisierten Abwägung von juristischen Argumenten oder im Bereich der künstlichen Intelligenz eingesetzt [9, 1f.] [15] [24, S. 1] [2, S. 620–624]. Dabei stellt sich oft die Frage, welches von zwei Argumenten stärker ist. Dieses Problem wird als **STRONGERDIS** bezeichnet [6, S. 3]. Die diskussions-basierte Semantik von Amgoud und Ben-Naim [1] liefert eine Heuristik zur Beantwortung dieser Frage, die die Anzahl von Angriffen auf ein Argument berücksichtigt. In oben stehendem Beispiel ist Argument 1 stärker als Argument 2, denn Argument

1 wird nur von einem Argument (2) angegriffen, während Argument 2 von zwei Argumenten (1 und 3) angegriffen wird.

3.2 Methodologie

Für das Experiment werden die folgenden Eingabegrößen, d. h. Anzahl der Argumente im Argumentations-Framework, mit jeweils 500 Testinstanzen pro Eingabegröße getestet: 10, 50, 100, 500, 1000, 5000. Argumentations-Frameworks mit mehr als 5000 Argumenten haben für die Speicherung der Adjazenzmatrix eine Speicherkomplexität von $O(5000^2)$, was bei der Ausführung auf meinem MacBook (M2 Chip, 16GB RAM) zu einem Heap-Overflow-Fehler führt. Die Testinstanzen werden durch den `DungTheoryGenerator` der `Tweety-Project-Bibliothek` [23] zufällig generiert. Anschließend werden zwei Zufallszahlen erzeugt, die den zu vergleichenden Argumenten entsprechen. Die Testinstanz, bestehend aus Adjazenzmatrix des Argumentations-Frameworks und den zwei Zufallsargumenten, wird als csv-Datei gespeichert. Dies erlaubt einen fairen Vergleich der verschiedenen Implementierungen, da sie auf den gleichen Testinstanzen ausgeführt werden.

Der Laufzeitanalyse in Java ist eine Warmlauf-Phase von 10 Iterationen pro Testinstanz vorgeschaltet, um dem JIT-Compiler Zeit zu geben, für den Algorithmus zu optimieren. Anschließend wird die eigentliche Messung durchgeführt¹. Da Julia ebenfalls über einen JIT-Compiler verfügt, wird für die Laufzeitanalyse der Julia-Implementierungen ebenfalls eine Warmlaufiteration pro Testinstanz durchgeführt, bevor die eigentliche Laufzeitmessung erfolgt.

¹In Absprache mit dem Betreuer meiner Bachelorarbeit wurde auf den Einsatz eines Benchmark-Tools verzichtet, da meine Analysen nicht im Nanosekundenbereich arbeiten, in denen solche Tools unverzichtbar sind. Des Weiteren war aus rein pragmatischen Gründen die verfügbare Zeit (3 Monate) für meine Bachelorarbeit nicht ausreichend für die 1000-fache Ausführung einer einzigen Testinstanz zur Bildung eines robusten Medians, wie z. B. mit der `Java Microbenchmark Harness Bibliothek`, da einige Testinstanzen alleine bereits 3 Stunden liefen und die 1000-fache Ausführung mehrere Monate gedauert hätte. Aus Gründen der Vergleichbarkeit wurde in den anderen Programmiersprachen ebenfalls auf den Einsatz eines spezifisches Benchmark-Tools verzichtet.

Der Rust- und C++-Code wurde automatisiert von Codex erzeugt. Der Prompt hierfür war, den Julia-Code nach Rust bzw. C++ zu übersetzen, einmal nur unter Verwendung der Standardbibliotheken der jeweiligen Sprache und einmal unter Verwendung einer BLAS-Bibliothek (Basic Linear Algebra Subprograms).

Zusammen mit den Laufzeiten wird für jede Testinstanz das Ergebnis des Algorithmus (true/false) gespeichert, um eine spätere Ergebnisvalidierung zu ermöglichen. Alle Implementierungen, Testinstanzen und -ergebnisse können auf folgender GitHub-Seite gefunden werden: github.com/vanessakobke/BachelorThesis.

3.3 Algorithmen zur Lösung von StrongerDis

Blümel et al. stellen einen Algorithmus zur Lösung von STRONGERDIS vor, der auf Matrix-Matrix-Multiplikation beruht [6, S. 7]. Dieser ist in Algorithmus 1 skizziert.

Algorithmus 1 StrongerDis_MM [6, S. 7]

Input: Argumentations-Framework $F = (A, R)$, Argumente $a, b \in A$

Output: **True** wenn Argument a stärker ist als Argument b , sonst **false**

```

1:  $i \leftarrow 1$ 
2:  $M \leftarrow \text{Adjazenzmatrix}(F)$ 
3: while  $i \leq 2|A| - 1$  do
4:   if  $i$  is odd  $\wedge M[a] < M[b]$  then return true
5:   if  $i$  is even  $\wedge M[b] < M[a]$  then return true
6:   if  $M[b] \neq M[a]$  then return false
7:    $M \leftarrow M \cdot \text{Adjazenzmatrix}(F)$ 
8:    $i \leftarrow i + 1$ 

```

Im Rahmen meiner Bachelorarbeit [16, 16ff.] und basierend auf Kiefer [14, S. 6] habe ich Algorithmus 2 entwickelt, der Matrix-Vektor-Multiplikation nutzt.

Beide Algorithmen beruhen auf dem Vergleich der Spaziergänge der Länge $i \in [1, 2|A|]$. Ein Spaziergang ungerade Länge entspricht einem Angriff auf das Argument a , wohingegen ein Spaziergang gerader Länge einer Verteidigung von Argument a entspricht [16, S. 7–8] [1, S. 142–143]. Die Anzahl der Spaziergänge

Algorithmus 2 StrongerDis_MV [16, S. 22–24]**Input:** Argumentations-Framework $F = (A, R)$, Argumente $a, b \in A$ **Output:** **True** wenn Argument a stärker ist als Argument b , sonst **false**

```

1:  $M \leftarrow \text{Adjazenzmatrix}(F)$ 
2:  $v_1 \leftarrow e_a, v_2 \leftarrow e_b$   $\# e_x$  steht für den Einheitsvektor, dessen  $x$ -te Komponente 1 ist
3: for  $i \leftarrow 1$  to  $2|A|$  do
4:   wähle eine zufällige Zahl  $r \in \{1, \dots, 2K|A|\}$ 
5:    $v_1 \leftarrow rMv_1$ 
6:    $v_2 \leftarrow rMv_2$ 
7:   if  $i$  is odd  $\wedge \text{sum}(v_1) < \text{sum}(v_2)$  then return true
8:   if  $i$  is even  $\wedge \text{sum}(v_2) < \text{sum}(v_1)$  then return true
9: return false

```

der Länge i können aus der Adjanzmatrix M^i abgelesen werden. Die Spaltensumme von Spalte a entspricht dann der gesamten Anzahl der Spaziergänge der Länge i , die in Argument a enden. Beide Algorithmen vergleichen nun die Anzahl der Spaziergänge gerader bzw. ungerader Länge für die Argumente a und b , um zu bestimmen, welches Argument stärker ist. Der Unterschied zwischen beiden Algorithmen besteht darin, wie sie die Spalten a und b von M^i berechnen. Während Algorithmus 1 dem naiven Verfahren der iterativen Matrixmultiplikationen folgt ($O(n^4)$), nutzt Algorithmus 2 aus, dass Matrixmultiplikationen spaltenweise erfolgen, und berechnet dadurch in jeder Iteration nur die Spalten a und b von M^i ($O(n^3)$) anstatt der gesamten Matrix [16, S. 23–24].

3.4 Julia-Implementierungen

Algorithmus 1 wurde in Julia implementiert wie in Abbildung 13 dargestellt. Hervorzuheben ist in dieser Implementierung, dass keine Nested-For-Loops nötig sind, um die Matrix-Matrix-Multiplikation oder die Spaltensummen für M_a und M_b zu berechnen. Julia bringt von Haus aus einfache Syntax und vorimplementierte Funktionalitäten mit, um diese Operationen umzusetzen. Insgesamt benötigt diese Implementierung in Julia 15 Code-Zeilen, während der entsprechende Java-Code unter Verwendung der EJML-Bibliothek für effiziente Matrixoperationen 20

```
1 function StrongerDis_MM(F, a, b)
2     M = copy(F)
3     for i in 1:(2*size(F,1) - 1)
4         Ma = sum(@view M[:, a])
5         Mb = sum(@view M[:, b])
6         if (i % 2 != 0 && Ma < Mb) || (i % 2 == 0 && Mb <
7             Ma)
8             return true
9         end
10        if Ma != Mb
11            return false
12        end
13        M = M * F;
14    end
15    return false
end
```

Abbildung 13: Julia-Implementierung von Algorithmus 1

Code-Zeilen benötigt. Eine Implementierung in der Java-Standardbibliothek ohne Verwendung von EJML würde Nested-For-Loops und damit noch mehr Code erfordern.

Algorithmus 2 wurde in Julia implementiert wie in Abbildung 14 dargestellt. Diese Implementierung benötigt 22 Code-Zeilen, wobei zu beachten ist, dass insbesondere die end-Zeilen, die Zeilenanzahl erhöhen. Die entsprechende Java-Implementierung (siehe Abbildung 21) benötigt 34 Zeilen.

Die einzige Optimierung, die in den Julia-Algorithmen verwendet wurde, ist das in Abschnitt 2.4 vorgestellte @view-Makro für die Erstellung der Spaltensumme in Algorithmus 1. Für alle anderen Matrixoperationen wurden keine expliziten Optimierungs- oder Parallelisierungsstrategien verwendet. Dies soll dazu dienen, zu prüfen, wie performant idiomatischer Julia-Code ist.

Da Julias Geschwindigkeit u. a. von den Compiler-Optimierungen abhängt, die mit Typenannotation zusammenhängen [5, S. 3], wurde für beide Implementierungen eine weitere Methode erstellt, deren Signatur vollständig typannotiert ist. Für Algorithmus 1 wurde die Signatur (F::MatrixFloat64, a::Int,

```
1 function strongerDis_MV(F, a, b)
2     n = size(F, 1)
3     v1 = zeros(Float64, n)
4     v2 = zeros(Float64, n)
5     v1[a] = 1.0
6     v2[b] = 1.0
7     for iter in 1:(2n)
8         r = rand(1:(2n*1000))
9         v1 = F * v1 * r
10        v2 = F * v2 * r
11        sumV1 = sum(v1)
12        sumV2 = sum(v2)
13        if abs(sumV1 - sumV2) > 1e-9
14            if isodd(iter)
15                return sumV1 > sumV2 ? false : true
16            else
17                return sumV1 > sumV2 ? true : false
18            end
19        end
20    end
21    return false
22 end
```

Abbildung 14: Julia-Implementierung von Algorithmus 2

$b::\text{Int}$) gewählt, da in diesem Algorithmus die Eingabematrix iterativ mit sich selbst multipliziert wird und die darin enthaltenen Werte damit sehr groß werden können. Für Algorithmus 2 wurde die Signatur $(F::\text{MatrixInt}, a::\text{Int}, b::\text{Int})$ gewählt, da in diesem Fall die Matrix unveränderlich und lediglich als Eingabe für die Matrix-Vektor-Multiplikation dient.

3.5 Ergebnisvalidierung

Um sicherzustellen, dass die Julia-Implementierungen fehlerfrei sind und die gleichen Ergebnisse liefern wie die Java-Implementierungen, wurde zu jeder Testinstanz das Ergebnis gespeichert und verglichen. Für diesen Zweck wurde der in Abbildung 15 dargestellte Julia-Code geschrieben, der die Ergebnisspalte aus den

```
1 using CSV
2 using DataFrames
3
4 java = CSV.read("results/java_results.csv", DataFrame)
5 julia = CSV.read("results/julia_results.csv", DataFrame)
6 df = innerjoin(java, julia, on="file", makeunique=true)
7 # Ergebnisvergleich
8 for row in eachrow(df)
9     if row.result != row.result_1
10         println("Fehler in Datei: ", row.file)
11     end
12 end
13
14 println("Vergleich abgeschlossen")
```

Abbildung 15: Julia-Code für die Ergebnisvalidierung

Java- und Julia-Ausgabedateien vergleicht. Die Ausführung dieses Codes auf den Ausgabedateien bestätigt, dass die verschiedenen Implementierungen korrekte Ergebnisse liefern. Die Ergebnisvalidierung für den Rust- und C++-Code wurde von Codex direkt in den jeweiligen Sprachen umgesetzt und bestätigt ebenfalls die Korrektheit der jeweiligen Implementierungen.

3.6 Laufzeitergebnisse und Diskussion

Wie erwartet zeigen die Ergebnisse, dass die Julia-, Rust- und C++-Implementierungen in den meisten Fällen deutlich schneller sind als die Java-Implementierungen. Abbildung 16 zeigt den Laufzeitvergleich für Algorithmus 1 und Abbildung 18 für Algorithmus 2. Da die verschiedenen Kurven sehr dicht beieinander liegen, stellen Abbildungen 17 und 19 zusätzlich die relativen Laufzeiten im Vergleich zur Java-Implementierung dar.

Interessant zu beobachten ist, dass die explizite Typpannotation in den Julia-Implementierungen fast keinen sichtbaren Einfluss auf die Geschwindigkeit hat. Bei kleiner Eingabegröße von 10 Argumenten ist die annotierte Methode etwas schneller als die unannotierte. Bei größeren Eingaben ist kaum Verbesserung

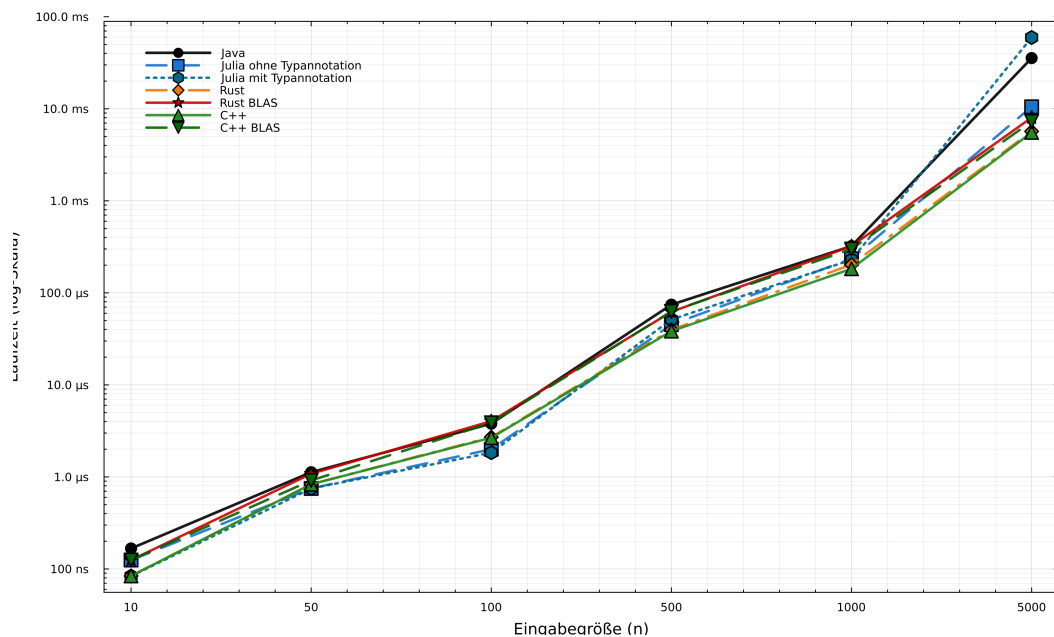


Abbildung 16: Laufzeitvergleich für Algorithmus 1

sichtbar.

Eine mögliche Erklärung in Anlehnung an [5, S. 11] ist, dass der Julia-Compiler bei diesem wenig komplexen Algorithmus durch eine Datenflussanalyse herleiten kann, mit welchen Datentypen die Funktionen aufgerufen werden und entsprechend optimieren. Eine Ausnahme tritt allerdings bei Algorithmus 2 mit Eingabegröße 1000 auf, in der die unannotierte Methode schneller ist als die annotierte. Dies scheint nicht auf einen Zufallseffekt zurückzuführen sein, da der selbe Verlauf bei dreimaliger Wiederholung des Experiments konstant blieb. Dies könnte möglicherweise an der Verwendung suboptimaler Datentypen in der annotierten Implementierung liegen. Eine genaue Analyse dieses Phänomens würde allerdings den Rahmen dieser Arbeit sprengen.

Überraschend ist, dass die BLAS-Implementierungen in Rust und C++, die nur zu dem Zweck hinzugefügt wurden, die Laufzeit optimieren, in Algorithmus 1 einen negativen Effekt haben. Dies ist insbesondere verwunderlich, da BLAS-Bibliotheken dafür gedacht sind, Operationen der linearen Algebra, wie Matrix-

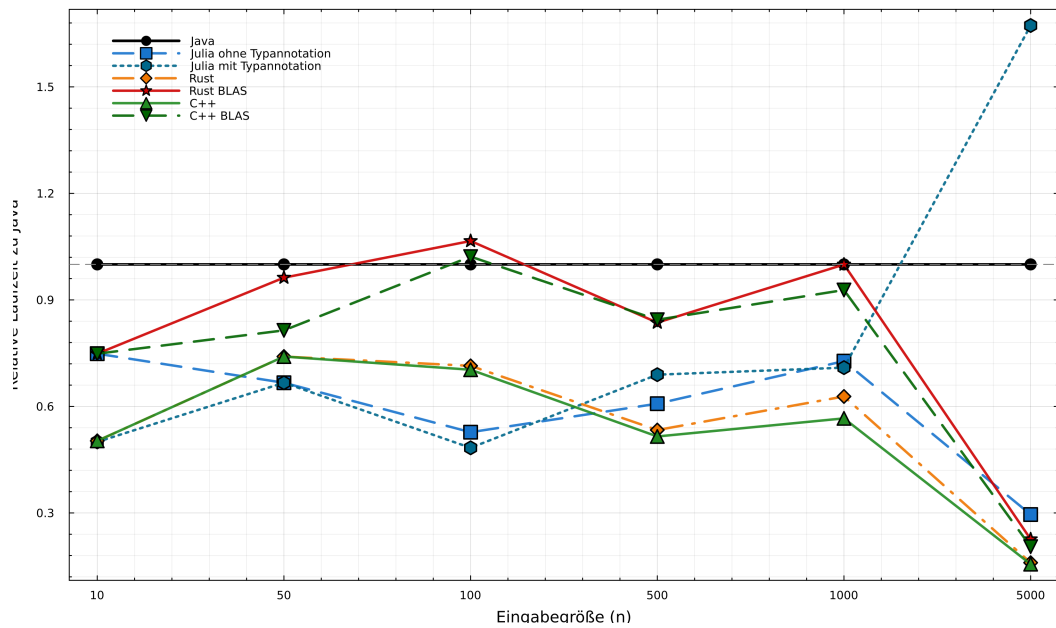


Abbildung 17: Relative Laufzeit für Algorithmus 1

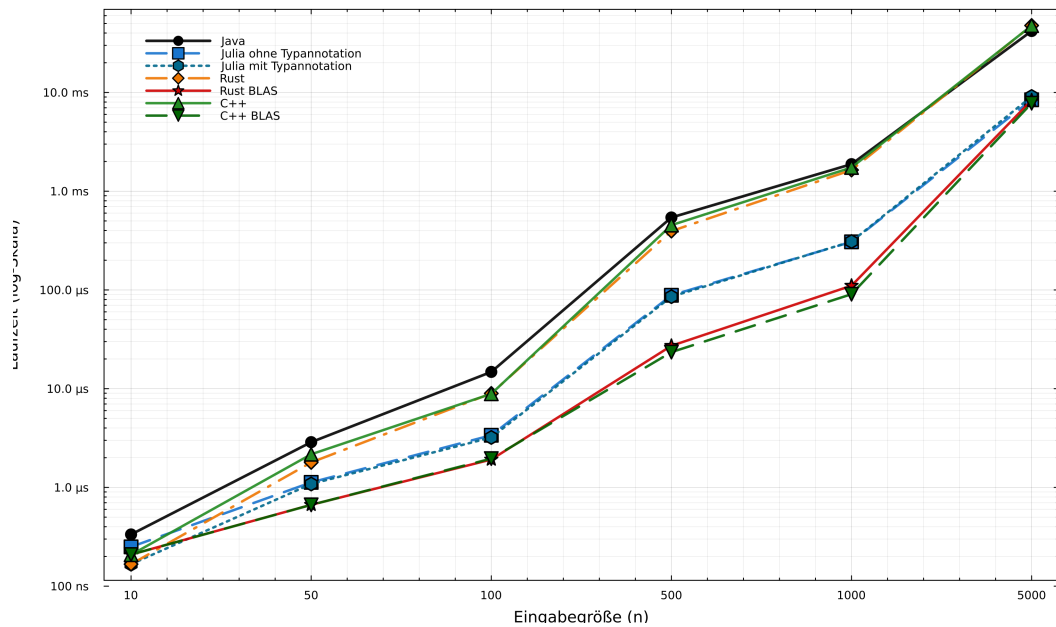


Abbildung 18: Laufzeitvergleich für Algorithmus 2

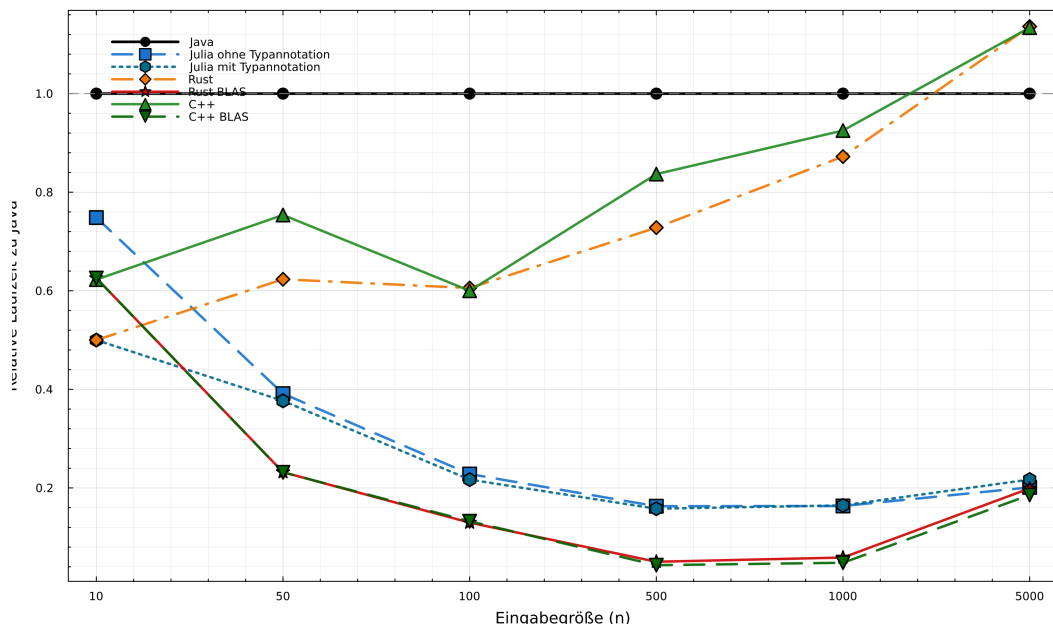


Abbildung 19: Relative Laufzeit für Algorithmus 2

Matrix-Multiplikationen zu beschleunigen. Tatsächlich führt dies für Algorithmus 1 teilweise sogar dazu, dass die Rust- und C++-Implementierungen mit BLAS langsamer sind als die Java-Implementierung. Einen gegenteiligen Effekt lässt sich für den auf Matrix-Vektor-Multiplikation basierenden Algorithmus 2 beobachten. Hier öffnet sich die Schere ab der Eingabegröße 50 sehr deutlich; während die Standard-Implementierungen bestenfalls halb so schnell sind, wie die Java-Implementierung und für die Eingabegröße 5000 sogar langsamer, benötigen die BLAS-Implementierungen nur circa ein Zehntel der Java-Laufzeit. Diese Diskrepanz zwischen Standard- und BLAS-Implementierungen in Rust und C++ ist eine interessante Beobachtung und bedarf weiterer Analyse.

Das durchgeführte Experiment zeigt, dass Julia in den betrachteten Algorithmen eine hohe Performanz erreicht und sich hinsichtlich der Laufzeit mit den untersuchten Implementierungen in Rust und C++ messen kann. Dabei zeigte sich zudem, dass sowohl der konkrete Algorithmus als auch die verwendete Implementierung und numerische Bibliothek einen erheblichen Einfluss auf die Laufzeit haben können. Einschränkend muss allerdings erwähnt werden, dass die hier un-

tersuchten Algorithmen eher eine Herausforderung bzgl. Speicherkomplexität als bzgl. Laufzeitkomplexität darstellen; während die Laufzeiten sich im Sekundenbereich bewegen, sind die Grenzen des Hauptspeichers meines MacBooks bereits ab einer Eingabegröße von 7000 Argumenten erreicht. Interessant wäre daher ein ähnlicher Vergleich der genannten Programmiersprachen anhand eines Algorithmus, dessen Herausforderung mehr in seiner Laufzeitkomplexität als in seiner Speicherkomplexität liegt.

4 Fazit und Ausblick

Diese Arbeit hat einen Überblick über die noch relativ junge Programmiersprache Julia gegeben. In Abschnitt 2 wurden die wichtigsten Eigenschaften von Julia betrachtet. Dabei stechen ihre dynamische Typisierung, die Nutzung des Multiple-Dispatch-Paradigmas und v. a. die Performanz hervor. Julia wird außerdem mit ihrer hohen Produktivität und Einsteigerfreundlichkeit beworben. Hinsichtlich der Erlernbarkeit und des Programmierkomforts deckt sich meine persönliche Erfahrung nur teilweise mit den in der Literatur hervorgehobenen Vorzügen von Julia. Zwar empfand ich das Erlernen der Sprache im Vergleich zu meinen bisherigen Erfahrungen mit Java, Python und Go nicht als schwieriger, jedoch auch nicht als merklich einfacher. Insbesondere das von Julia angestrebte Skriptsprachen-Look-and-Feel und die damit verbundene Flexibilität haben sich für mich während der praktischen Beschäftigung mit der Sprache nicht in dem Maße bemerkbar gemacht, wie ich es aufgrund der beschriebenen Eigenschaften erwartet hätte. Beispielsweise akzeptiert Julia (anders als Java) bei Methodensignaturen nur exakte Übereinstimmungen und führt keine automatische Konvertierung beispielsweise von `Int64` zu `Int` durch. Dabei handelt es sich ausdrücklich um eine subjektive Einschätzung, die nicht als allgemeine Aussage über die Erlernbarkeit von Julia verstanden werden kann.

Insgesamt scheint Julia eine aufgrund ihrer Geschwindigkeit interessante Programmiersprache zu sein, die den Programmierer nicht mit einer komplexen Syntax einschränkt. Allerdings hat der Vergleich mit Java-Implementierungen im

Bereich der linearen Algebra und Verwendung der Java-Bibliothek EJML gezeigt, dass Julia zwar schneller, aber keine Größenordnungen schneller ist. Die Performance ist vergleichbar mit Sprachen wie C++ und Rust.

Interessante Möglichkeiten für zukünftige Forschung sind der Einsatz von Julia in Projekten, die sich üblicherweise für objektorientierte Programmierung anbieten, wie die Entwicklung komplexer Anwendungsprogramme, sowie die Exploration von Julias Fähigkeiten zur Programmierung von graphischen Benutzerschnittstellen.

Literatur

- [1] Leila Amgoud und Jonathan Ben-Naim. „Ranking-Based Semantics for Argumentation Frameworks“. In: *Scalable Uncertainty Management*. Hrsg. von David Hutchison u. a. Bd. 8078. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, S. 134–147. ISBN: 978-3-642-40380-4 978-3-642-40381-1. DOI: 10.1007/978-3-642-40381-1_11. (Besucht am 25.03.2026).
- [2] T.J.M. Bench-Capon und Paul E. Dunne. „Argumentation in artificial intelligence“. en. In: *Artificial Intelligence* 171.10-15 (Juli 2007), S. 619–641. ISSN: 00043702. DOI: 10.1016/j.artint.2007.05.001. (Besucht am 26.03.2026).
- [3] *BenchmarkTools.jl - Manual*. URL: <https://juliaci.github.io/BenchmarkTools.jl/stable/manual/>.
- [4] Jeff Bezanson u. a. *Julia: A Fresh Approach to Numerical Computing*. arXiv:1411.1607 [cs.MS]. Juli 2015. DOI: 10.48550/arXiv.1411.1607. URL: <http://arxiv.org/abs/1411.1607> (besucht am 28.05.2026).
- [5] Jeff Bezanson u. a. „Julia: dynamism and performance reconciled by design“. en. In: *Proceedings of the ACM on Programming Languages* 2.OOPSLA (Okt. 2018), S. 1–23. ISSN: 2475-1421. DOI: 10.1145/3276490. URL: <https://dl.acm.org/doi/10.1145/3276490> (besucht am 17.03.2026).
- [6] Lydia Blümel u. a. *On the Complexity of the Discussion-based Semantics in Abstraction Argumentation*. Version Number: 1. 2026. DOI: 10.48550/ARXIV.2604.11480. (Besucht am 14.04.2026).
- [7] Tyler A. Cabutto u. a. „An Overview of the Julia Programming Language“. en. In: *Proceedings of the 2018 International Conference on Computing and Big Data*. Charleston SC USA: ACM, Sep. 2018, S. 87–91. ISBN: 978-1-4503-6540-6. DOI: 10.1145/3277104.3277119. URL: <https://dl.acm.org/doi/10.1145/3277104.3277119> (besucht am 17.03.2026).

- [8] Benjamin Chung. „A Type System for Julia“. PhD Thesis. Northeastern University, 2023.
- [9] Phan Minh Dung. „On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games“. en. In: *Artificial Intelligence* 77.2 (Sep. 1995), S. 321–357. ISSN: 00043702. DOI: 10.1016/0004-3702(94)00041-X. (Besucht am 26.03.2026).
- [10] M N Gevorkyan u. a. „Statistically significant performance testing of Julia scientific programming language“. In: *Journal of Physics: Conference Series* 1205 (Apr. 2019), S. 012017. ISSN: 1742-6588, 1742-6596. DOI: 10.1088/1742-6596/1205/1/012017. URL: <https://iopscience.iop.org/article/10.1088/1742-6596/1205/1/012017> (besucht am 17.03.2026).
- [11] GitHub. *GitHub Innovation Graph - Programming Languages*. Juli 2026. URL: <https://innovationgraph.github.com/global-metrics/programming-languages>.
- [12] Jane Herman. *Intro to Julia tutorial (version 1.0)*. Nov. 2018. URL: https://www.youtube.com/watch?v=8h8rQyEpiZA&list=PLP8iPy9hna6SCcFv3FvY_qjAmtTsNYHQE&index=9.
- [13] Tomasz Januszek und Mariusz Pleszczyński. „Comparative analysis of the efficiency of Julia language against the other classic programming languages“. In: *Silesian Journal of Pure and Applied Mathematics* Vol. 8, iss. 1 (2018), S. 49–56.
- [14] Stefan Kiefer u. a. „On the Complexity of Equivalence and Minimisation for Q-weighted Automata“. en. In: *Logical Methods in Computer Science* Volume 9, Issue 1 (März 2013), S. 908. ISSN: 1860-5974. DOI: 10.2168/LMCS-9(1:8)2013. URL: <https://lmcs.episciences.org/908> (besucht am 26.05.2026).
- [15] Robert A. Kowalski und Francesca Toni. „Abstract argumentation“. en. In: *Artificial Intelligence and Law* 4.3-4 (1996), S. 275–296. ISSN: 0924-8463, 1572-8382. DOI: 10.1007/BF00118494. (Besucht am 25.03.2026).

- [16] Vanessa Köbke. „NC Complexity and Comparison of Algorithms for the Discussion-based Semantics in Abstract Argumentation“. Bachelor’s thesis. Hagen: FernUniversität in Hagen, Aug. 2026. URL: https://github.com/vanessakoebe/BachelorThesis/blob/main/documentation/Bachelors_thesis.pdf.
- [17] Soumen Pal u. a. „A next-generation dynamic programming language Julia: Its features and applications in biological science“. en. In: *Journal of Advanced Research* 64 (Okt. 2024), S. 143–154. ISSN: 20901232. DOI: 10.1016/j.jare.2023.11.015. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2090123223003521> (besucht am 17.03.2026).
- [18] A Poetzsch-Heffter. *63611 Einführung in die objektorientierte Programmierung*. Studienmaterialien. Hagen: FernUniversität in Hagen, 2025.
- [19] Julia Project. *The Julia Programming Language*. 2026. URL: <https://julialang.org/>.
- [20] Julia Project. *Views (SubArrays and other view types)*. 2026. URL: [https://docs.julialang.org/en/v1/base/arrays/#Views-\(SubArrays-and-other-view-types\)](https://docs.julialang.org/en/v1/base/arrays/#Views-(SubArrays-and-other-view-types)).
- [21] StackOverflow. *StackOverflow Developer Survey 2025*. Juli 2026. URL: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language-language-write-ins>.
- [22] TIOBE. *TIOBE Index for May 2026*. 2026. URL: <https://www.tiobe.com/tiobe-index/>.
- [23] *Tweety Project Version 1.30*. 2026. URL: <https://tweetyproject.org/>.
- [24] Douglas Walton. „Argumentation Theory: A Very Short Introduction“. en. In: *Argumentation in Artificial Intelligence*. Hrsg. von Guillermo Simari und Iyad Rahwan. Boston, MA: Springer US, 2009, S. 1–22. ISBN: 978-0-387-98196-3 978-0-387-98197-0. DOI: 10.1007/978-0-387-98197-0_1. (Besucht am 29.03.2026).

- [25] Francesco Zappa Nardelli u. a. „Julia subtyping: a rational reconstruction“. en. In: *Proceedings of the ACM on Programming Languages* 2.OOPS-LA (Okt. 2018), S. 1–27. ISSN: 2475-1421. DOI: 10 . 1145 / 3276483. URL: <https://dl.acm.org/doi/10.1145/3276483> (besucht am 26.05.2026).

A Anhang

A.1 Sequenzielle Java-Implementierung von Algorithmus 1 mit der EJML-Bibliothek

```
1 import org.ejml.simple.SimpleMatrix;
2
3 public static boolean strongerDis_Sequential(SimpleMatrix
4     F, int a, int b) {
5     SimpleMatrix M = F.copy();
6     for (int i = 1; i <= 2 * F.getNumRows() - 1; i++)
7     {
8         int Ma = 0;
9         int Mb = 0;
10        for (int j = 0; j < F.getNumRows(); j++) {
11            Ma += M.get(j, a);
12            Mb += M.get(j, b);
13        }
14        if ((i % 2 != 0 && Ma < Mb) || (i % 2 == 0 &&
15            Mb < Ma)) {
16            return true;
17        }
18        if (Ma != Mb) {
19            return false;
20        }
21        M = M.mult(F);
22    }
23    return false;
24 }
```

Abbildung 20: Java-Implementierung von Algorithmus 1

A.2 Sequenzielle Java-Implementierung von Algorithmus 2 mit der EJML-Bibliothek

```
1 import org.ejml.simple.SimpleMatrix;
2 public static boolean strongerDis_Sequential(SimpleMatrix
   F, int a, int b) {
3     int num_arguments = F.getNumRows();
4     SimpleMatrix v1 = new SimpleMatrix(num_arguments,
       1);
5     SimpleMatrix v2 = new SimpleMatrix(num_arguments,
       1);
6     v1.set(a, 0, 1);
7     v2.set(b, 0, 1);
8     for (int iter = 1; iter <= 2*num_arguments; iter
       +++) {
9         SimpleMatrix v1New =F.mult(v1);
10        SimpleMatrix v2New =F.mult(v2);
11        v1 = v1New;
12        v2 = v2New;
13        double sumV1 = v1.elementSum();
14        double sumV2 = v2.elementSum();
15        if (Math.abs(sumV1 - sumV2) > 1e-9) {
16            if (iter % 2 != 0) {
17                if (sumV1 > sumV2) {
18                    return false;
19                } else {
20                    return true;
21                }
22            }
23            if (iter % 2 == 0) {
24                if (sumV1 > sumV2) {
25                    return true;
26                } else {
27                    return false;
28                }
29            }
30        }
31    }
32    return false;
33 }
```

Abbildung 21: Java-Implementierung von Algorithmus 2